An index in Elasticsearch is like a table in a database

In a relational database, we define a table to hold our records. Extending this concept, an index in Elasticsearch is equivalent to a table in a database; Elasticsearch allows us to create a schema-free index (meaning we don't need to create a schema based on our data model beforehand, unlike in a database, where a table cannot exists without a schema), which helps with indexing free-form text.

An *index* is a logical collection holding documents in shards. *Shards* are running instances of Apache Lucene. (Understanding the mechanics is a rabbit hole and may seem overwhelming—hang on tight, because we cover shards and other moving parts shortly.)

TYPES OF DATA

Data comes in various types: dates, numbers, strings, Booleans, IP addresses, locations, and so on. Elasticsearch lets us index documents by supporting rich data types. A process called *mapping* converts JSON data types into appropriate Elasticsearch data types. Mapping uses a schema definition to let Elasticsearch know how to handle our data fields. For example, the `title` and `synopsis` fields in the news article document in listing 3.1 are text fields, while `published_date` is a date field and `number_of_words` is a numeric field.

While indexing data, Elasticsearch analyzes incoming data field by field. It analyzes each field using advanced algorithms based on mapping definitions. It then stores these fields in efficient data structures such that the data is in a form to be searched and analyzed for our purposes. Full-text fields undergo an additional process called *text analysis*, which is the heart and soul of a modern search engine like Elasticsearch. Text analysis (discussed in the next section) is a crucial step in getting our raw data into a form that enables Elasticsearch to retrieve data efficiently while supporting a myriad of queries.

ANALYZING DATA

Data represented as text is analyzed during the text analysis phase. The text is broken down into words (called *tokens*) using a set of rules. Fundamentally, two processes happen during the analysis process: tokenization and normalization.

Tokenization is the process of breaking text into tokens based on a set of rules. As shown in table 3.1, text in the `synopsis` field is broken down into individual words (tokens) separated by a whitespace delimiter (this is done by a `standard` tokenizer—a built-in software component that breaks text into tokens based on predefined rules).

Table 3.1 Untokenized vs. tokenized `synopsis` field

Untokenized `String` (before tokenization)	`Peter Piper picked a peck of Pickled-peppers!`
Tokenized `String` (after tokenization)	`[peter,piper,picked,a,peck,of,pickled,peppers]` Notes: the tokens were lowercased, and the hyphen and exclamation point were removed.